

Advantages:-

- Very fast access.
- Best for large symbol tables.

Disadvantages:-

- Collision handling needed.
- Performance depends on hash function.

c) Binary Search Tree (BST):-

- Stores elements in sorted order.
- $\text{Left} < \text{Root} < \text{Right}$.

Time Complexity:-

- Search $\rightarrow O(\log n)$ (balanced).
- Insert $\rightarrow O(\log n)$.
- Delete $\rightarrow O(\log n)$.

Advantages:-

- Maintains sorted order.
- Efficient searching (if balanced).

Disadvantages:-

- Can degrade to $O(n)$ if unbalanced.
- More complex than hash table.

Hash Table is the most efficient because:-

- it provides constant time $O(1)$ operations in average case.
- fast lookup is critical in compilers.
- widely used in real compiler implementations.

3) Design of Hash Table:-

- Table size $(m) = 10$
- Hash Function:-

$h(\text{key}) = \text{length of identifier mod } 10.$

- Collision Handling:- Chaining (Linked List).